

```

import java.io.*;
import java.util.*;
import java.util.regex.*;

public class Pass2Assembler {

    static class Symbol {
        String name;
        int address;
        Symbol(String n, int a) { name = n; address = a; }
    }

    static class Literal {
        String literal;
        int address;
        Literal(String l, int a) { literal = l; address = a; }
    }

    static Map<Integer, Symbol> symtab = new HashMap<>();
    static Map<Integer, Literal> littab = new HashMap<>();

    public static void main(String[] args) throws Exception {
        if (args.length < 4) {
            System.out.println("Usage: java Pass2Assembler <intermediate> <symtab> <littab> <pooltab>");
            return;
        }

        String intermediateFile = args[0];
        String symFile = args[1];
        String litFile = args[2];
        loadSymtab(symFile);
        loadLittab(litFile);

        generateMachineCode(intermediateFile);

        System.out.println("Machine code written to machine.txt");
    }

    static void loadSymtab(String file) throws IOException {
        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("Index")) continue;
                String[] parts = line.split("\\s+");
                if (parts.length >= 3) {
                    try {
                        int index = Integer.parseInt(parts[0]);
                        String name = parts[1];
                        int addr = Integer.parseInt(parts[2]);
                        symtab.put(index, new Symbol(name, addr));
                    } catch (NumberFormatException e) {
                    }
                }
            }
        }
    }

    static void loadLittab(String file) throws IOException {
        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
            }
        }
    }
}

```

```

        if (line.isEmpty() || line.startsWith("Index")) continue;
        String[] parts = line.split("\\s+");
        if (parts.length >= 3) {
            try {
                int index = Integer.parseInt(parts[0]);
                String literal = parts[1];
                int addr = Integer.parseInt(parts[2]);
                littab.put(index, new Literal(literal, addr));
            } catch (NumberFormatException e) {
            }
        }
    }
}

static void generateMachineCode(String file) throws IOException {
    BufferedWriter bw = new BufferedWriter(new FileWriter("machine.txt"));
    bw.write("ADDR\tMACHINE_CODE\n");

    Pattern instrPat = Pattern.compile("\\((\\w+), (\\d+)\\)");

    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = br.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty()) continue;

            String[] parts = line.split("\\s+");
            String addr = parts[0];
            StringBuilder machine = new StringBuilder();

            Matcher m = instrPat.matcher(line);
            if (m.find()) {
                String type = m.group(1);
                String code = m.group(2);

                if (type.equals("IS")) {
                    machine.append(code).append(" ");
                    for (String p : parts) {
                        if (p.matches("\\d+")) {
                            machine.append(p).append(" ");
                        } else if (p.startsWith("(S,") ) {
                            int index = Integer.parseInt(p.substring(3, p.length() -
1));

                            Symbol sym = symtab.get(index);
                            if (sym != null) {
                                machine.append(sym.address).append(" ");
                            } else {
                                machine.append("0 ").append(" ");
                            }
                        } else if (p.startsWith("(L,") ) {
                            int index = Integer.parseInt(p.substring(3, p.length() -
1));

                            Literal lit = littab.get(index);
                            if (lit != null) {
                                machine.append(lit.address).append(" ");
                            } else {
                                machine.append("0 ").append(" ");
                            }
                        }
                    }
                } else if (type.equals("DL") && code.equals("01")) {
                    if (parts.length > 2) {
                        machine.append("00 0 ").append(parts[2]);
                    } else {

```

```

        machine.append("00 0 0");
    }
}

    bw.write(addr + "\t" + machine.toString().trim() + "\n");
}

    bw.close();
}
}

```

Output:-

--MACHINE CODE--

ADDR	MACHINE_CODE
200	
200	04 200 0
201	05 201
202	04 202
203	04 203
204	01 204 206
205	07 205 202
206	
206	00 0 5
207	00 0 1
208	02 208 0
209	07 209 208
210	00 210
211	
204	03 204
205	
211	
212	
212	
213	
213	00 0 1